DECODO Automation Integration Guide

Overview

This guide provides comprehensive documentation for the enhanced DECODO integration in Eiros Agent iOS, featuring full automation capabilities including screenshot capture, JavaScript rendering, and advanced web scraping.

🚀 New Features

Enhanced Capabilities

1. **Screenshot Capture (PNG)**
   - Headless browser screenshots via `X-SU-Headless: png`
   - Web Scraping API screenshot support
   - Integration with Qwen2VL vision model
   - Automatic image analysis and processing

2. **JavaScript Rendering**
   - Dynamic content rendering via `X-SU-Headless: html`
   - Support for both Site Unblocker and Web Scraping API
   - Real-time page interaction capabilities

3. **Advanced Automation**
   - Full automation task system
   - Multi-step workflow execution
   - Error handling and retry logic
   - Progress tracking and logging

4. **Geo-Targeting & Sessions**
   - Location-based request routing
   - Session management for consistent IPs
   - Custom headers and parameters
   - Device type emulation

🏗️ Architecture

Core Components

```
DECODOConfiguration
├── Site Unblocker Configuration
├── Web Scraping API Configuration
├── Credential Management
├── Connection Testing
└── Header/Payload Generation

ProxyManager
├── Proxy Configuration Management
├── Connection Health Monitoring
├── Screenshot Capture Integration
├── Automation Request Handling
└── Statistics & Monitoring

AutomationController
├── Task Creation & Execution
├── Action Processing
├── Progress Tracking
├── Log Management
└── Vision Integration

UI Components
├── DECODOSettingsView
├── AutomationPanelView
├── Task Creator
└── Results Display
```

📋 Configuration

1. DECODO Credentials Setup

```
// Set credentials
DECODOConfiguration.shared.setCredentials(
    username: "your_username",
    password: "your_password"
)

// Verify configuration
let isConfigured = DECODOConfiguration.shared.isConfigured
```

2. Site Unblocker Configuration

```
// Get Site Unblocker configuration
let config = DECODOConfiguration.shared.getSiteUnblockerConfiguration()

// Configuration includes:
// - Endpoint: unblock.decodo.com:60000
// - Credentials for authentication
// - Capabilities: JavaScript, Screenshots, Geo-targeting
```

3. Web Scraping API Configuration

```
// Get Web Scraping API configuration
let config = DECODOConfiguration.shared.getWebScrapingAPIConfiguration()

// Configuration includes:
// - Endpoint: https://scraper-api.decodo.com/v2/scrape
// - Advanced parsing capabilities
// - Target templates support
```

🖼️ Screenshot Capture

Site Unblocker Screenshots

```
// Create headers for PNG screenshot
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    headlessMode: .png,
    geoLocation: "United States",
    sessionId: "session123"
)

// Capture screenshot through proxy
let result = await ProxyManager.shared.captureScreenshotThroughProxy(
    url: "https://example.com",
    options: ScreenshotOptions(
        geoLocation: "United States",
        deviceType: .desktop
    )
)

if result.success, let imageData = result.imageData {
    let image = UIImage(data: imageData)
    // Process screenshot
}
```

Web Scraping API Screenshots

```
// Create payload for screenshot
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "https://example.com",
    target: "universal",
    headlessMode: .png,
    geoLocation: "Germany",
    deviceType: .desktop
)

// Execute via Web Scraping API
// Returns JSON with base64-encoded screenshot
```

🤖 Automation Tasks

Creating Screenshot Tasks

```
// Create screenshot task
let task = AutomationController.shared.createScreenshotTask(
    url: "https://example.com",
    options: ScreenshotTaskOptions(
        geoLocation: "United States",
        deviceType: .desktop,
        sessionId: "session123"
    )
)

// Execute task
let result = await AutomationController.shared.executeTask(task)

if result.success {
    // Process screenshots
    for screenshotData in result.screenshots {
        let image = UIImage(data: screenshotData)
        // Analyze with Qwen2VL
    }
}
```

Creating Web Scraping Tasks

```
// Create web scraping task
let task = AutomationController.shared.createWebScrapingTask(
    url: "https://example.com",
    selectors: [".title", ".price", ".description"],
    options: WebScrapingTaskOptions(
        geoLocation: "Germany",
        deviceType: .mobileAndroid,
        extractAttribute: "text",
        extractMultiple: true
    )
)

// Execute task
let result = await AutomationController.shared.executeTask(task)

if result.success, let data = result.data {
    let extractedData = try JSONSerialization.jsonObject(with: data)
    // Process extracted data
}
```

Custom Automation Tasks

```
// Create custom task with multiple actions
let actions = [
    AutomationAction(
        id: UUID().uuidString,
        type: .navigate,
        target: "https://example.com",
        parameters: ["geoLocation": "United States"]
    ),
    AutomationAction(
        id: UUID().uuidString,
        type: .wait,
        target: "",
        parameters: ["duration": "3.0"]
    ),
    AutomationAction(
        id: UUID().uuidString,
        type: .screenshot,
        target: "https://example.com",
        parameters: [:]
    ),
    AutomationAction(
        id: UUID().uuidString,
        type: .extract,
        target: ".main-content",
        parameters: ["attribute": "text"]
    )
]

let task = AutomationTask(
    id: UUID().uuidString,
    name: "Custom Automation",
    description: "Navigate, wait, screenshot, and extract",
    actions: actions,
    options: AutomationTaskOptions(
        useProxy: true,
        enableJavaScript: true,
        timeout: 120.0
    )
)
```

🌍 Geo-Targeting & Localization

Site Unblocker Geo-Targeting

```
// Country-based targeting
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    geoLocation: "Germany"
)

// City-based targeting
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    geoLocation: "Berlin, Germany"
)

// State-based targeting
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    geoLocation: "California, United States"
)

// Coordinate-based targeting
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    geoLocation: "lat: 40.7128, lng: -74.0060, rad: 10000"
)
```

Web Scraping API Geo-Targeting

```
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "https://example.com",
    geoLocation: "United Kingdom",
    locale: "en-GB",
    deviceType: .desktop
)
```

📱 Device Emulation

Supported Device Types

```
// Desktop emulation
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "https://example.com",
    deviceType: .desktop
)

// Mobile Android emulation
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "https://example.com",
    deviceType: .mobileAndroid
)

// Mobile iOS emulation
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "https://example.com",
    deviceType: .mobileIOS
)
```

🔧 Advanced Features

Session Management

```
// Create persistent session
let sessionId = UUID().uuidString

// Use session across multiple requests
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    sessionId: sessionId
)

// Session remains active for 10 minutes
```

Custom Headers

```
// Add custom headers
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    customHeaders: [
        "User-Agent": "Custom User Agent",
        "Accept-Language": "de-DE,de;q=0.9",
        "Referer": "https://google.com"
    ]
)

// Headers are prefixed with X-SU-Custom-
// Result: X-SU-Custom-User-Agent, X-SU-Custom-Accept-Language, etc.
```

Status Code Handling

```
// Accept custom status codes
let headers = DECODOConfiguration.createSiteUnblockerHeaders(
    statusCodes: [200, 404, 503]
)

// Useful for scraping error pages or maintenance pages
```

POST Requests

```
// Web Scraping API POST request
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "https://httpbin.org/post",
    httpMethod: .POST,
    payload: Data("{\"key\": \"value\"}".utf8)
)
```

🎯 Target Templates

Using Pre-built Templates

```
// Google Search template
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "artificial intelligence",
    target: "google_search",
    geoLocation: "United States",
    locale: "en-US"
)

// Amazon Product template
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "B08N5WRWNW", // ASIN
    target: "amazon_product",
    geoLocation: "United States"
)

// Universal template (default)
let payload = DECODOConfiguration.createWebScrapingAPIPayload(
    url: "https://example.com",
    target: "universal"
)
```

🔍 Vision Integration

Automatic Screenshot Analysis

```
// Screenshots are automatically analyzed with Qwen2VL
let task = AutomationController.shared.createScreenshotTask(
    url: "https://example.com"
)

let result = await AutomationController.shared.executeTask(task)

// Vision analysis results are stored in the automation session
// and can be accessed for further processing
```

Manual Vision Analysis

```
// Capture screenshot
let screenshotResult = await ProxyManager.shared.captureScreenshotThroughProxy(
    url: "https://example.com",
    options: ScreenshotOptions()
)

if let imageData = screenshotResult.imageData,
   let image = UIImage(data: imageData) {

    // Analyze with Vision Manager
    let visionResult = await VisionManager.shared.analyzeImage(image)

    // Process vision results
    print("Detected objects: \(visionResult.detectedObjects)")
    print("Extracted text: \(visionResult.text)")
}
```

📊 Monitoring & Statistics

Connection Monitoring

```
// Get current proxy information
let proxyInfo = ProxyManager.shared.getCurrentProxyInfo()

print("Proxy: \(proxyInfo?.name ?? "None")")
print("Latency: \(proxyInfo?.formattedLatency ?? "N/A")")
print("Data transferred: \(proxyInfo?.formattedDataTransferred ?? "N/A")")
print("Uptime: \(proxyInfo?.formattedUptime ?? "N/A")")
```

Automation Status

```
// Get automation status
let status = AutomationController.shared.getAutomationStatus()

print("Automation enabled: \(status.isEnabled)")
print("Current task: \(status.currentTask ?? "None")")
print("Execution status: \(status.executionStatus.description)")
print("Progress: \(Int(status.progress * 100))%")
print("Proxy enabled: \(status.proxyEnabled)")
print("Components ready: \(status.componentsReady)")
```

Execution Logs

```
// Access execution logs
let logs = AutomationController.shared.executionLog

for entry in logs.suffix(10) {
    print("[\(entry.timestamp)] \(entry.type.rawValue.uppercased()): \
(entry.message)")
}
```

🧪 Testing

Connection Testing

```
// Test Site Unblocker connection
let result = await DECODOConfiguration.shared.testSiteUnblockerConnection()

if result.success {
    print("✅ Site Unblocker connected - Latency: \(result.formattedLatency)")
} else {
    print("❌ Site Unblocker failed: \(result.error ?? "Unknown error")")
}

// Test Web Scraping API connection
let apiResult = await DECODOConfiguration.shared.testWebScrapingAPI()

if apiResult.success {
    print("✅ Web Scraping API connected - Latency: \(apiResult.formattedLatency)")
} else {
    print("❌ Web Scraping API failed: \(apiResult.error ?? "Unknown error")")
}
```

Unit Testing

```
// Run comprehensive tests
// See DECODOIntegrationTests.swift for full test suite

// Test configuration
XCTAssertTrue(DECODOConfiguration.shared.hasValidCredentials())

// Test proxy manager
XCTAssertTrue(ProxyManager.shared.isProxyEnabled)

// Test automation controller
XCTAssertTrue(AutomationController.shared.isAutomationEnabled)
```

🚨 Error Handling

Common Error Scenarios

```
// Handle connection errors
let result = await ProxyManager.shared.captureScreenshotThroughProxy(
    url: "https://example.com",
    options: ScreenshotOptions()
)

if !result.success {
    switch result.error {
    case "No proxy enabled":
        // Enable proxy first
        await ProxyManager.shared.enableDECODOSiteUnblocker()
    case "DECODO not configured":
        // Configure credentials
        DECODOConfiguration.shared.setCredentials(username: "user", password: "pass")
    default:
        // Handle other errors
        print("Screenshot failed: \(result.error ?? "Unknown error")")
    }
}
```

Automation Error Handling

```
// Handle task execution errors
let result = await AutomationController.shared.executeTask(task)

if !result.success {
    print("Task failed: \(result.error ?? "Unknown error")")

    // Check execution logs for details
    let errorLogs = AutomationController.shared.executionLog.filter { $0.type == .er-
ror }
    for log in errorLogs.suffix(5) {
        print("Error: \(log.message)")
    }
}
```

🔐 Security Considerations

Credential Management

- Credentials are stored securely in iOS Keychain
- Passwords are never exposed in logs or UI
- Automatic credential cleanup on app uninstall

Network Security

- All connections use HTTPS/TLS encryption
- Proxy authentication via secure headers
- No sensitive data in URL parameters

Data Privacy

- Screenshots and extracted data are processed locally

- No data is sent to third parties without explicit consent
- Automatic cleanup of temporary files

📈 Performance Optimization

Best Practices

1. **Session Reuse**
   ```swift
   // Reuse sessions for multiple requests
   let sessionId = UUID().uuidString
   // Use same sessionId for related requests
   ```

2. **Efficient Task Design**
   ```swift
   // Combine related actions in single task
   // Use appropriate timeouts
   // Enable JavaScript only when needed
   ```

3. **Resource Management**
   ```swift
   // Clean up large screenshots after processing
   // Limit concurrent automation tasks
   // Monitor memory usage
   ```

🔄 Migration from Previous Version

Breaking Changes

1. **Configuration Structure**
   - Old: Mock endpoints (`proxy.decodo.io`)
   - New: Real endpoints (`unblock.decodo.com`)

2. **Authentication**
   - Old: No authentication required
   - New: Username/password required from DECODO dashboard

3. **Capabilities**
   - Old: Basic proxy functionality
   - New: Full automation with screenshots and JavaScript

Migration Steps

1. **Update Configuration**
   ```swift
   // Remove old mock configuration
   // Add real DECODO credentials
   DECODOConfiguration.shared.setCredentials(username: "real_user", password: "real_pass")
   ```

2. **Update Code**
   ```swift
   // Replace old proxy calls with new automation tasks
   ```

```
   // Use new screenshot capture methods
   // Implement error handling for real API responses
```

3. **Test Integration**
   ```
   swift
   // Test with real DECODO credentials
   // Verify screenshot capture works
   // Test automation workflows
   ```

📚 API Reference

DECODOConfiguration

- `setCredentials(username:password:)` - Set DECODO credentials
- `getSiteUnblockerConfiguration()` - Get Site Unblocker config
- `getWebScrapingAPIConfiguration()` - Get Web Scraping API config
- `testSiteUnblockerConnection()` - Test Site Unblocker connection
- `testWebScrapingAPI()` - Test Web Scraping API connection
- `createSiteUnblockerHeaders(...)` - Generate Site Unblocker headers
- `createWebScrapingAPIPayload(...)` - Generate Web Scraping API payload

ProxyManager

- `enableDECODOSiteUnblocker()` - Enable Site Unblocker proxy
- `enableDECODOWebScrapingAPI()` - Enable Web Scraping API proxy
- `captureScreenshotThroughProxy(url:options:)` - Capture screenshot
- `executeAutomationThroughProxy(request:)` - Execute automation
- `getCurrentProxyInfo()` - Get current proxy information
- `testProxyConnection(_:)` - Test proxy connection

AutomationController

- `createScreenshotTask(url:options:)` - Create screenshot task
- `createWebScrapingTask(url:selectors:options:)` - Create scraping task
- `executeTask(_:)` - Execute automation task
- `getAutomationStatus()` - Get automation status
- `enableAutomation()` - Enable automation
- `disableAutomation()` - Disable automation

🆘 Troubleshooting

Common Issues

1. **“DECODO not configured”**
   - Solution: Set valid credentials in DECODOSettingsView

2. **“No proxy enabled”**
   - Solution: Enable proxy in AutomationPanelView or programmatically

3. **“Connection failed”**
   - Check internet connectivity

- Verify DECODO service status
- Test credentials in DECODO dashboard

4. **“Screenshot capture failed”**
   - Ensure proxy supports screenshot capability
   - Check target URL accessibility
   - Verify geo-location settings

5. **“Task execution timeout”**
   - Increase task timeout in options
   - Check network latency
   - Simplify task complexity

Debug Information

```
// Enable debug logging
// Check execution logs
let logs = AutomationController.shared.executionLog

// Test individual components
let configTest = await DECODOConfiguration.shared.testSiteUnblockerConnection()
let proxyTest = await ProxyManager.shared.testProxyConnection(proxy)

// Check component status
let status = AutomationController.shared.getAutomationStatus()
print("Components ready: \(status.componentsReady)")
```

📞 Support

For issues related to DECODO integration:

1. **Configuration Issues**: Check DECODOSettingsView and credentials
2. **Connection Problems**: Test with DECODO dashboard first
3. **Automation Failures**: Review execution logs and error messages
4. **Performance Issues**: Monitor statistics and optimize task design

For DECODO service issues, contact DECODO support directly through their dashboard.

This comprehensive guide covers all aspects of the enhanced DECODO integration. The system now provides full automation capabilities with screenshot capture, JavaScript rendering, and advanced web scraping features, all integrated seamlessly with the Eiros Agent iOS application.